



SCHOOL OF PROGRAMMING AND DEVELOPMENT

Software Architect

Syllabus

UDACITY.COM

Overview

This program equips learners with essential software engineering skills, from writing reliable, well-tested code to designing scalable, maintainable, and robust systems. It emphasizes proven design and architecture approaches alongside a systems-level perspective, preparing learners to build high-quality software solutions that perform effectively in real-world environments.

Nanodegree Program



Intermediate



53 hours



5.0 (1 Reviews)

Prerequisites

Prior to enrolling, you should have the following knowledge:

Digital Project Management Fluency

API Awareness

Object-oriented Python

Basic github

You will also need to be able to communicate fluently and professionally in **written and spoken English**.

Skills You'll Learn

Test-Driven Development

Pytest | Scaling Test Strategies | Test Code Maintainability | Structuring Test Suites | Software Test Types | Test-driven development | Interpreting Test Output | Debugging via Test Failures | Red-Green-Refactor Workflow

Design Patterns

Adapter Pattern | Command Pattern | Structural design patterns | Strategy Pattern | Creational design patterns | Decorator Pattern | Design pattern fundamentals | Behavioral design patterns | Factory Pattern | Composite Pattern | Builder Pattern | Observer Pattern | Singleton Pattern | Dependency injection

Software Architecture Patterns

Caching and CDN Architecture | Multitier architecture | Data Storage Tradeoffs in Architecture | Microservices architecture basics | IoT System Architecture | IT architecture diagramming | Serverless cloud computing architecture | Software architecture | Cloud-Native Architecture | Blockchain System Architecture | Model-view-controller | Machine learning system architectures

Systems Engineering

System Models | System Verification | Requirement Validation and Verification | Togaf framework | System Integration | Systems Engineering Core Principles | Software development lifecycle | SysML | System Validation | Zachman framework | Architectural Frameworks | Systems thinking | Systems Engineering Stages | ReqView | Requirements gathering | Requirements Management | System Design Principles | Requirements Analysis



Courses

01. Test-Driven Development

14 hours

Gain confidence in your software with Test-Driven Development (TDD). Practice the red-green-refactor loop with Python and pytest—writing tests before you write your code. Learn how unit, integration, and end-to-end tests fit together to provide a comprehensive safety net. Organize clean test suites using advanced techniques: fixtures, mocking, and parameterization. Deploy with confidence at scale by incorporating testing into CI/CD and pre-flight checklists. Leverage TDD to use AI tools the right way—encode your requirements as tests, then let AI help implement the code that passes them. By the end of this course, you will be able to refactor with confidence, catch regressions early, and ship features faster with less risk.

L1	Introduction to Test-Driven Development	
L2	Introduction to Testing in Software Development	Learn why TDD exists, how it fits into modern software practices, and what types of tests (unit, integration, end-to-end) support a robust testing culture.
L3	Writing Unit Tests in Python	Gain hands-on experience using pytest to write simple, test-first Python code. Follow the arrange-act-assert framework for unit testing. Map tests to CRUD operations—create, read, update, delete.
L4	Creating Robust and Maintainable Test Suites	Apply best practices to name, group, and organize tests. Utilize advanced pytest techniques, including fixtures, parameterization, and mocking.
L5	Diagnosing Failures and Scaling Testing in Practice	Diagnose failures: read errors, debug with breakpoints, and find root causes. Then go practical—write integration tests against real APIs, track coverage, and use pre-flight checklists before release.
L6	Project: Udatracker Order Management System	Implement a web application—including backend business logic and a Flask API—using test-driven development.

02. Design Patterns

11 hours

Build a strong foundation in object-oriented design by applying proven software design patterns in Python. Explore creational, structural, and behavioral patterns, and see how each category addresses common challenges in building flexible, reusable, and scalable systems. Learn why these patterns go beyond "common sense" and represent decades of accumulated engineering wisdom. Practice translating abstract ideas into clear, maintainable

code. By the end of this course, you will understand how design patterns connect to SOLID principles, dependency injection, and modern AI-assisted development workflows.

L1	Introduction to Design Patterns	Learn what design patterns are, why they exist, and how they connect to SOLID principles to help you design maintainable, testable, and scalable software.
L2	Creational Design Patterns	Apply creational patterns—Singleton, Factory Method, and Builder—to design flexible solutions where object creation should be separated from usage.
L3	Structural Design Patterns	Use Adapter, Decorator, and Composite patterns in Python to modify or combine objects without changing their internal logic.
L4	Behavioral Design Patterns	Learn how Observer, Strategy, and Command patterns support delegation, flexible behavior, and decoupled object interactions.
L5	Principles in Practice	Apply dependency injection to decouple objects, support testing, and enable clean, flexible implementations of modern design patterns.
L6	Project: Personal Finance Manager for Freelancers and Gig Workers	Apply classic design patterns to a working Python finance application, combining required patterns with one of your choice and validating your design through testing.

03. Software Architecture Patterns

15 hours

Design systems that scale beyond a single codebase. Learn how to think like a software architect by exploring proven architectural patterns, cloud-native design, and real-world system tradeoffs. Model applications using monolithic and event-driven architectures, diagram complex systems, and understand the foundations of microservices and serverless computing. Design data pipelines and storage strategies, reason about machine learning system architecture, and make informed choices around caching, CDNs, IoT, and blockchain systems. By the end, you'll be equipped to move from "working code" to well-designed, resilient architectures fit for modern software systems.

L1	Introduction to Software Architecture Patterns	
L2	Introduction to Software Architecture	Define what software architecture is and why it matters. Explore foundational patterns, including MVC, layered, and event-driven architectures, to build intuition for structuring real-world systems.
L3	Communicating Software Architecture	Turn architectural ideas into clear diagrams. Learn when to use sequence, event flow, and system diagrams—and how to create them with Mermaid and D2.

L4	Microservices and Modular Design Patterns	Design modular systems with microservices. Explore when to split systems apart, how services communicate, and the tradeoffs around resilience, consistency, and performance.
L5	Cloud-Native Architectural Patterns	Don't treat the cloud as an afterthought. Design cloud-native systems using 12-factor principles, stateless services, scalable architectures, and cost-efficient patterns.
L6	Data and AI Architecture Patterns	Design data, machine learning, and AI systems by choosing the right storage, pipelines, and architectures—covering batch vs. streaming data, model serving, orchestration, and observability.
L7	Distributed Architecture Patterns	Explore distributed system patterns through CDNs, IoT networks, and blockchain architectures, learning how specialized constraints shape real-world architectural decisions.
L8	Project: Real-Time Financial Fraud Detection - Solution Design	Design a real-time, cloud-native fraud detection architecture for a bank, balancing low latency, high throughput, compliance, observability, and elastic scaling using modern architectural patterns.

04. Systems Engineering

13 hours

This course introduces Systems Engineering principles, focusing on the lifecycle of complex systems. You will explore key concepts such as system design, architecture, requirements analysis, modeling, and verification. The course covers various system lifecycle models, including Waterfall, V-Model, Spiral, and Agile. You'll also learn essential tools like SysML for system modeling, risk management, and trade-off analysis. In the final project, you will design and validate a Smart Home Security System, documenting requirements, evaluating design trade-offs, and creating system models. You will also develop a verification and validation plan to ensure the system meets user needs.

L1	Introduction to Systems Engineering	Systems engineering is an approach to design, develop and manage complex systems across their lifecycle, focusing on defining customer needs and then requirements, design, implementation and testing.
L2	Requirements Analysis and System Modeling	Requirement analysis define what a system must do, while system modeling visualizes structure and behavior to ensure clear design, traceability, and successful development.
L3	System Design and Architecture	System design and architecture focus on defining system structure, evaluating design trade-offs, and documenting architecture using models to guide development, integration, and performance evaluation
L4	System Integration, Verification, and Validation	System integration combines components into a working system, while verification ensures it meets requirements and validation confirms it satisfies user needs through testing and continuous validation

L5

Project:

Design and Validation of a Smart Home Security System

Design and validate a Smart Home Security System with sensors, cameras, alarms and a control unit. Document requirements, evaluate trade-offs, model with SysML and develop a verification plan.

Meet Your Instructors



Laura Morinigo

Software Engineer and AI Strategist

Laura is a Software Engineer & AI Strategist driving innovation at the forefront of emerging technologies. She combines technical expertise with strategic vision to shape responsible, ethical AI solutions and influence the future of technology globally.



Afreen Aliya

Engineering Manager at Wealthsimple

Afreen Aliya is a fintech-focused Software Developer and Team Lead who loves translating complex engineering ideas into clear, practical lessons for learners at all levels.



Liam Stevens

Cloud & Platform Engineering Consultant
at Culture Amp

Liam Stevens is a cloud infrastructure and platform engineering specialist with over eight years of experience designing, building, and scaling production systems across AWS and Google Cloud. Liam has led platform teams and delivered multi-cloud solutions for enterprise, startup and SaaS companies globally.



José Cano

Software Developer at Globant

José Cano is a software developer at Globant, where he works for a gaming company. He previously worked as an IoT developer focusing on creating embedded devices for monitor and remote controlling applications. He is very passionate about DIY projects and creating his own devices to perform different tasks.

Why Udacity



Demonstrate proficiency with practical projects

Projects are based on real-world scenarios and challenges, allowing you to apply the skills you learn to practical situations, while giving you real hands-on experience

- ✓ Gain proven experience
- ✓ Retain knowledge longer
- ✓ Apply new skills immediately



24/7 access to real human support

Reviewers provide timely and constructive feedback on your project submissions, highlighting areas of improvement and offering practical tips to enhance your work

- ✓ Get help from subject matter experts
- ✓ Gain valuable insights and improve your skills
- ✓ Learn industry best practices